

컴퓨터프로그램저작물 등록신청서

추가 알고리즘 명세 — 사용자 주도형 경주 시뮬레이션 수동 보정 모듈

기본 사항 (기존 등록 정보와 동일)

항목	내용
저작물명	경마통 Race Master — 실시간 AI 경마 레이스 시뮬레이터
추가 알고리즘명	사용자 주도형 경주 시뮬레이션 실시간 배팅 배당률 연동 수동 보정 엔진
개발 언어	Dart 3.9.2 / Flutter 3.35.4
운영환경	Android 15 (API 35), Web(Chrome/Firefox/Safari)
저작물 유형	컴퓨터프로그램저작물 (소스코드 + UI 레이아웃 명세)

제1조 알고리즘 개요 및 창작 배경

본 알고리즘은 기존 등록된 「경마통 Race Master」의 물리 엔진(GridRail 4존 엔진 및 Jockey Engine)에, **사용자가 UI를 통해 개별 경주마의 물리 파라미터를 실시간으로 수정·주입(inject)할 수 있는 인터페이스 레이어**를 추가한 독립적 컴퓨터프로그램저작물이다.

기존 경마 시뮬레이터는 KRA(한국마사회) 공공 API에서 수신한 데이터를 기반으로 자동 연산하는 구조였으나, 본 모듈은 **배당률 가중치, 기수 멘탈 상태, 스포츠 시점**이라는 3가지 핵심 변수를 유저가 직접 보정할 수 있는 「수동 변수 보정 패널(Manual Calibration Panel)」과, 이 입력값을 물리 엔진의 P_{final} 다중 멀티플라이어 체인에 실시간으로 바인딩하는 「API 파라미터 오버라이딩 제어 계층(UserCalibrationController)」을 새롭게 창작하였다.

제2조 UI 레이아웃 구조 명세

2-1. 수동 변수 보정 패널 평면 배치

수동 변수 보정 패널은 기존 배당률 표(Odds Table)와 동일 화면에 나란히 배치되는 독립 컨테이너(Container)로 구성되며, 다음과 같은 계층적 위젯 트리 구조를 가진다.

```

ManualCalibrationPanel (StatelessWidget)
|
├ Container (배경: #0E1E3A, 테두리: Gold #33FFD700, radius: 16px)
|   └ Column (crossAxisAlignment: stretch)
|       |
|       └ [1] _PanelHeader (Row)
|           └ Icon(tune_rounded) + Text('수동 변수 보정 패널')
|           └ 활성 보정수 배지 (Container: Gold #FFD700, '${N}마 보정중')
|               └ TextButton('전체 초기화') ← ctrl.resetAll() 트리거
|
|   └ Divider (구분선)
|
|   └ [2] _HorseCalibTile × N마 (마별 보정 행, Column)
|       └ _GateNoChip - 마번 번호 칩 (색상 순환 10색)
|       └ CalibrationOddsField - ① 배당률 가중치 입력창
|       └ CalibrationJockeySlider - ② 기수 멘탈 보정 슬라이더
|       └ CalibrationSpurtDropdown - ③ 스퍼트 시점 드롭다운
|
|   └ [3] _PanelFooter (Row)
|       └ Icon(check_circle / lock)
|       └ Text('보정값 실시간 반영 중 · 마지막 업데이트 HH:MM:SS')

```

2-2. 개별 UI 컴포넌트 평면 배치

각 `_HorseCalibTile` 은 수직(Column) 레이아웃으로 아래 3개 컴포넌트를 순서대로 배치한다:

```

_HorseCalibTile (gateNo=N)
├─ Row [마번칩][말이름·배당·기수명][초기화아이콘]
│
├─ CalibrationOddsField (Row)
│   ├─ Text('배당 가중치', width: 80)
│   └─ TextFormField (TextInputType.numberWithOptions)
│       └─ 입력 범위: -2.0 ~ +2.0 (소수점 1자리)
│           └─ inputFormatters: [FilteringTextInputFormatter]
│               └─ 실시간 색상: 양수=초록(#66BB6A), 음수=빨강(#EF5350)
│                   └─ onChanged → ctrl.setOddsWeight(gateNo, value)
│                       └─ Text('±2.0')
│
├─ CalibrationJockeySlider (Row)
│   ├─ Text('기수 멘탈', width: 80)
│   └─ Text('패널티')
│       └─ Slider (min:-1.0, max:+1.0, divisions:20)
│           └─ SliderTheme (activeTrackColor: 동적 색상)
│               └─ onChanged → ctrl.setJockeyBuff(gateNo, value)
│                   └─ Text('버프')
│                       └─ Text('+N%' / '-N%' / '기본')
│
└─ CalibrationSpurtDropdown (Row)
    ├─ Text('스퍼트 시점', width: 80)
    └─ DropdownButton<SpurtTimingSlot>
        ├─ 초기 스퍼트 (-50m)
        ├─ 표준 (기본값)
        ├─ 후기 스퍼트 (+40m)
        └─ 극후기 (+80m)
            └─ onChanged → ctrl.setSpurtTiming(gateNo, slot)

```

2-3. 데이터 바인딩 트리 구조

UI 컴포넌트에서 물리 엔진까지의 완전한 데이터 바인딩 경로:

[사용자 입력]

```
CalibrationOddsField.onChange(value)
→ UserCalibrationController.setOddsWeight(gateNo, value)
→ RaceCalibrationSnapshot.update(HorseCalibration(userOddsWeight))
→ ChangeNotifier.notifyListeners()
→ Consumer<UserCalibrationController>.builder 리빌드 (16ms 이내)
```

[물리 엔진 반영 — 매 틱 16ms]

```
_initHorses():
base × ctrl.oddsAdjFactor(gateNo) → calibratedBase (baseSpeed 보정)
ctrl.spurtProgOffset(gateNo) → _horseSpurtOffset[gateNo]

_onTick() → inZone4Straight:
ctrl.speedMultOverride(gateNo, afScale) → speedMult × (1.0 ± Δ)

applyToEntry(entry):
newUserBonus = entry.userBonus
+ (userOddsWeight × 0.015 × 333) ← oddsAdj
+ (userJockeyBuff × 0.12 × 333) ← jockeyAdj
→ .clamp(-5.0, +5.0)
→ _Horse.userBonus → Zone3 speedMult *= 1.12 + bf×0.08×user
```

제3조 핵심 소스코드 명세 — user_calibration_controller.dart

3-1. 파일 구성 요약

파일명	위치	역할
user_calibration_model.dart	lib/models/	보정 데이터 DTO 계층
user_calibration_controller.dart	lib/providers/	물리 엔진 오버라이딩 제어
manual_calibration_panel.dart	lib/widgets/	UI 컴포넌트 집합

3-2. 핵심 데이터 모델 — SpurtTimingSlot 열거형

```
// 스퍼트 시점 드롭다운 선택값 → Zone4 진입 prog 오프셋으로 변환
enum SpurtTimingSlot {
  early ('조기 스퍼트', -0.05), // 기준보다 50m 이른 스퍼트 발동
  standard ('표준', 0.0), // KRA 기본 스퍼트 시점 (변경 없음)
  delayed ('후기 스퍼트', 0.04), // 기준보다 40m 늦은 스퍼트 발동
  veryLate ('극후기', 0.08); // 기준보다 80m 늦은 스퍼트 발동

  const SpurtTimingSlot(this.label, this.progOffset);
  final String label;
  final double progOffset; // prog 단위 오프셋 (0.01 ≈ 경주거리의 1%)
}
```

3-3. 핵심 데이터 모델 — HorseCalibration 불변 객체

```
// 마번 1개의 보정 데이터 집합 (불변 value object)
@immutable
class HorseCalibration {
  final int gateNo; // 마번 식별자 (1-based)
  final double userOddsWeight; // 배당률 가중치: -2.0 ~ +2.0 (기본 0.0)
  final double userJockeyBuff; // 기수 멘탈 보정: -1.0 ~ +1.0 (기본 0.0)
  final SpurtTimingSlot spurtTiming; // 스퍼트 시점 (기본 standard)

  bool get isDefault =>
    userOddsWeight == 0.0 &&
    userJockeyBuff == 0.0 &&
    spurtTiming == SpurtTimingSlot.standard;
}
```

3-4. 핵심 제어 메서드 — API 파라미터 오버라이딩 연산식

(1) setOddsWeight — 배당률 가중치 실시간 주입

```
void setOddsWeight(int gateNo, double value) {
  // ① 범위 검증 및 클램프 (-2.0 ~ +2.0)
  final clamped = value.clamp(kOddsWeightMin, kOddsWeightMax);
  // ② 불변 스냅샷 갱신 (Dart immutable update pattern)
  _snapshot = _snapshot.update(current.copyWith(userOddsWeight: clamped));
  // ③ ChangeNotifier 전파 → UI 즉시 갱신 + 물리 루프 동기화
  notifyListeners();
}
```

물리 엔진 반영 경로 (oddsAdjFactor):

```
 $\Delta factor = userOddsWeight \times kOddsWeightSpeedDelta(0.015)$ 
baseSpeed_final = baseSpeed_KRA  $\times (1.0 + \Delta factor)$ 
유효 범위: baseSpeed  $\times [0.970, 1.030]$ 
```

(2) setJockeyBuff — 기수 멘탈 슬라이더 실시간 주입

```
void setJockeyBuff(int gateNo, double value) {
  final clamped = value.clamp(kJockeyBuffMin, kJockeyBuffMax);
  _snapshot = _snapshot.update(current.copyWith(userJockeyBuff: clamped));
  notifyListeners();
}
```

물리 엔진 반영 경로 (speedMultOverride):

```
 $\Delta mult = userJockeyBuff \times kJockeyBuffMaxDelta(0.12) \times afScale(1.0 \sim 1.35)$ 
speedMult_Zone4  $\times= (1.0 + \Delta mult)$ 
유효 범위: speedMult  $\times [0.838, 1.162]$  (afScale=1.0 기준)
유효 범위: speedMult  $\times [0.784, 1.216]$  (afScale=1.35, 9경주 기준)
```

(3) setSpurtTiming — 스퍼트 시점 드롭다운 주입

```
void setSpurtTiming(int gateNo, SpurtTimingSlot slot) {
    _snapshot = _snapshot.update(current.copyWith(spurtTiming: slot));
    notifyListeners();
}
```

물리 엔진 반영 경로 (spurtProgOffset):

```
_horseSpurtOffset[gateNo] = slot.progOffset (-0.05 ~ +0.08)
```

적용 시점: `_initHorses()` — 경주 시작 전 1회 계산

효과:

- early(-0.05): 기본 스퍼트 시점보다 5% 일찍 가속 → 선행말 유리
- standard(0.0): 변경 없음
- delayed(+0.04): 기본보다 4% 늦게 가속 → 추입형 후방 배치마 유리
- veryLate(+0.08): 기본보다 8% 늦게 가속 → 극단적 후방 대기 전략

(4) applyToEntry — P_final 전체 오버라이딩 합산식

```
HorseEntry applyToEntry(HorseEntry entry) {
    if (calib.isDefault) return entry; // 보정 없으면 zero-cost 반환

    // ① 배당률 가중치 → userBonus 환산
    // userOddsWeight +2.0 → userBonus +10.0 (스케일: ×333)
    final oddsAdj = calib.userOddsWeight × kOddsWeightSpeedDelta × 333.0;

    // ② 기수 멘탈 슬라이더 → userBonus 환산
    // userJockeyBuff +1.0 → userBonus +40.0 (스케일: ×333)
    final jockeyAdj = calib.userJockeyBuff × kJockeyBuffMaxDelta × 333.0;

    // ③ 합산 후 물리 엔진 userBonus 범위(-5.0~+5.0) 클램프
    final newUserBonus = (entry.userBonus + oddsAdj + jockeyAdj).clamp(-5.0, 5.0);

    return entry.copyWith(userBonus: newUserBonus);
}
```

P_final 전체 연산 흐름 (사용자 보정 포함 완전판):

```
P_final = baseSpeed × oddsAdjFactor(userOddsWeight)
          × blockMult × cornerPenalty × boostMult × spurtMult
          × g1fBoostMult × safeModeCoeff × mentalBuffCoeff
          × surgeBuffCoeff × speedMultOverride(userJockeyBuff, afScale)
          + noise(±0.00002)
```

여기서:

- `oddsAdjFactor = 1.0 + userOddsWeight × 0.015 ∈ [0.970, 1.030]`
- `speedMultOverride = 1.0 + userJockeyBuff × 0.12 × afScale ∈ [0.784, 1.216]`

제4조 기능 명세서 — 실시간 배팅 배당률 연동 및 사용자 주도형 수동 보정

4-1. 기능 정의

기능명: 실시간 배팅 배당률 연동 사용자 주도형 경주 시뮬레이션 데이터 수동 보정 기능

기능 구분: KRA API 파라미터 오버라이딩(Parameter Overriding) 기반 실시간 물리 엔진 재조정 인터페이스

작동 원리:

1. KRA 공공 API로부터 수신한 배당률(odds), 기수 통산승률(jockeyRcWins), 마별 G1F 성적(g1fRating) 등의 원시 데이터를 기반으로 물리 엔진이 초기 파라미터를 산출한다.
2. 사용자가 「수동 변수 보정 패널」에서 다음 3종 변수를 직접 수정한다:
 - **userOddsWeight (배당률 가중치):** -2.0~+2.0 범위의 부동소수점 값. 배당률 기반 기초 속도 산출 계수를 $\pm 3\%$ 범위에서 오버라이드한다.
 - **userJockeyBuff (기수 멘탈 보정):** -1.0~+1.0 범위의 슬라이더 값. Zone4(후반 400m~결승선) 가속도 계수를 $\pm 12\% \times \text{afScale}$ 범위에서 오버라이드한다.
 - **spurtTiming (스퍼트 시점):** 4단계 드롭다운 선택값. Zone4 스퍼트 발동 prog 오프셋을 -5%~+8% 범위에서 개별 조정한다.
3. UserCalibrationController.notifyListeners()가 즉시 호출되어 물리 루프(16ms 주기)의 다음 틱에서 보정값이 반영된다.
4. 경주 시작 시점에 lockPanel()이 호출되어 보정 패널이 잠기며, 경주 종료 후 unlockPanel()로 해제된다.

4-2. API 파라미터 오버라이딩 기술 명세

오버라이딩 대상 API 파라미터 목록:

파라미터명	API 출처	기본 연산식	오버라이드 방식	유효 범위
baseSpeed	KRA rcWins, jockeyRcWins	$\text{rcWins} \times 50 + \text{jockeyRcWins} \times 30 + (60 - \text{wgBudam}) \times 0.2$	$\times \text{oddsAdjFactor}$	$\pm 3\%$
speedMult_Zone4	내부 계산	$\text{g1fBoostMult} \times \text{safeModeCoeff} \times \dots$	$\times \text{speedMultOverride}$	$\pm 12 \sim 16\%$
_spurt100	거리 기반 계산	$\text{goalP} - 100\text{m} / \text{총거리}$	$+ \text{spurtProgOffset}$	-5%~+8%
userBonus	KRA userBonusScore	원시 KRA 값	$+ (\text{oddsAdj} + \text{jockeyAdj})$	± 5.0

오버라이딩 적용 시점:

파라미터	적용 시점	적용 메서드
baseSpeed 보정	경주 시작 시 1회 (_initHorses)	oddsAdjFactor(gateNo)
spurt 오프셋	경주 시작 시 1회 (_initHorses)	spurtProgOffset(gateNo)
Zone4 speedMult	매 틱 16ms (_onTick)	speedMultOverride(gateNo, afScale)
userBonus 합산	매 틱 16ms (_onTick)	applyToEntry(entry)

4-3. ChangeNotifier 기반 리액티브 상태 관리 명세

```

UserCalibrationController (ChangeNotifier)
|
| RaceCalibrationSnapshot ← 불변(immutable) 보정 스냅샷
|   Map<int, HorseCalibration> → gateNo당 HorseCalibration 1개
|
|
| setOddsWeight() → notifyListeners() → Consumer 리빌드 (≤16ms)
| setJockeyBuff() → notifyListeners() → Consumer 리빌드 (≤16ms)
| setSpurtTiming() → notifyListeners() → Consumer 리빌드 (≤16ms)
|
|
| lockPanel() ← 경주 시작 시 호출 → panelEnabled = false
| unlockPanel() ← 경주 종료 시 호출 → panelEnabled = true

```

스레드 안전성: Flutter의 단일 스레드(Dart isolate) 모델에 의해 경쟁 조건(race condition) 없이 안전하게 동작한다. notifyListeners()는 위젯 rebuild 큐에 등록되며, 물리 루프(Timer.periodic)와 동일 isolate에서 순차 실행된다.

4-4. 창작성 진술

본 알고리즘의 독창적 창작 요소는 다음과 같다:

- 3변수 오버라이딩 구조의 독창성:** 배당률 가중치, 기수 멘탈, 스포트 시점이라는 서로 다른 물리적 의미를 가진 3가지 변수를 단일 HorseCalibration 불변 객체로 캡슐화하고, 이를 P_final 8중 멀티플라이어 체인의 서로 다른 위치에 선택적으로 주입하는 구조는 타 경마 시뮬레이터에 존재하지 않는 독자적 설계이다.
- 비선형 스케일 팩터(×333)의 창작성:** userOddsWeight(-2.0~+2.0)와 userJockeyBuff(-1.0~+1.0)를 userBonus(-5.0~+5.0) 스케일로 역변환하기 위해 333을 스케일 팩터로 채택한 것은, 기존 물리 엔진의 userBonus 입력 범위(-5~+5)와의 호환성을 유지하면서도 UI 입력 범위(±2.0, ±1.0)를 직관적으로 설계한 독창적 수치 설계이다.
- 패널 잠금/해제 메커니즘의 창작성:** 경주 시작(lockPanel) → 경주 종료(unlockPanel)의 생명주기 연동은, 경주 도중 파라미터 조작으로 인한 물리 엔진 불안정을 방지하는 동시에 결과 확인 후 즉시 다음 경주 보정을 준비할 수 있도록 설계된 창작적 상태 기계(state machine)이다.

4. **SpurtTimingSlot 열거형의 도박론적 설계**: 조기(-50m)/표준/후기(+40m)/극후기(+80m) 4단계 스퍼트 시점 분류는, 실제 경마에서 관찰되는 선행마·중간마·후방마의 전략적 행동 패턴을 prog 오프셋으로 추상화한 것으로, 현실 경마 데이터 분석에 기반한 독창적 모델링이다.
5. **Zero-cost 최적화 패턴의 창작성**: `if (calib.isDefault) return entry;` 구문으로, 보정값이 없는 경우 새 객체 생성을 완전히 생략하는 zero-cost 반환 패턴은, 16ms 물리 루프에서 발생하는 GC 압력을 최소화하기 위한 Flutter 특화 최적화 기법이다.
6. **불변 스냅샷 갱신 패턴(Immutable Update Pattern)의 적용**: Dart의 불변 객체 + copyWith 패턴을 활용하여, 보정 상태의 이전 스냅샷을 보존하면서 특정 마번만 선택적으로 갱신하는 `RaceCalibrationSnapshot.update()` 구조는 Flutter ChangeNotifier와 결합된 독창적 상태 관리 설계이다.

제5조 소스코드 추출본 전문

5-1. user_calibration_model.dart (핵심 발췌)

```
// — SpurtTimingSlot: 스퍼트 시점 열거형 —
enum SpurtTimingSlot {
  early ('조기 스퍼트', -0.05),
  standard('표준', 0.0),
  delayed ('후기 스퍼트', 0.04),
  veryLate('극후기', 0.08);

  const SpurtTimingSlot(this.label, this.progOffset);
  final String label;
  final double progOffset;
}

// — HorseCalibration: 마번 1개의 보정 데이터 집합 —
@immutable
class HorseCalibration {
  final int gateNo;
  final double userOddsWeight; // -2.0 ~ +2.0
  final double userJockeyBuff; // -1.0 ~ +1.0
  final SpurtTimingSlot spurtTiming;

  const HorseCalibration({
    required this.gateNo,
    this.userOddsWeight = 0.0,
    this.userJockeyBuff = 0.0,
    this.spurtTiming = SpurtTimingSlot.standard,
  });

  bool get isDefault =>
    userOddsWeight == 0.0 && userJockeyBuff == 0.0 &&
    spurtTiming == SpurtTimingSlot.standard;
}
```

5-2. user_calibration_controller.dart (핵심 발췌)

```
// — 물리 엔진 오버라이드 상수 —
const double kOddsWeightSpeedDelta = 0.015; // ±3% baseSpeed 보정
const double kJockeyBuffMaxDelta = 0.12; // ±12%×afScale speedMult 보정

// — API 파라미터 오버라이딩 핵심 메서드 —

/// [1] baseSpeed oddsAdjFactor: 배당률 가중치 → 기초속도 보정 계수
double oddsAdjFactor(int gateNo) {
  final calib = _snapshot.forGate(gateNo);
  if (calib.userOddsWeight == 0.0) return 1.0; // zero-cost
  return (1.0 + calib.userOddsWeight * kOddsWeightSpeedDelta).clamp(0.97, 1.03);
}

/// [2] Zone4 speedMultOverride: 기수 멘탈 슬라이더 → 가속도 계수 오버라이드
double speedMultOverride(int gateNo, double afScale) {
  final calib = _snapshot.forGate(gateNo);
  if (calib.userJockeyBuff == 0.0) return 1.0; // zero-cost
  final delta = calib.userJockeyBuff * kJockeyBuffMaxDelta * afScale;
  return (1.0 + delta).clamp(0.7, 1.35);
}

/// [3] spurtProgOffset: 스퍼트 시점 드롭다운 → Zone4 진입 오프셋
double spurtProgOffset(int gateNo) =>
  _snapshot.forGate(gateNo).spurtTiming.progOffset;

/// [4] applyToEntry: P_final 전체 오버라이딩 합산
HorseEntry applyToEntry(HorseEntry entry) {
  final calib = _snapshot.forGate(entry.gateNo);
  if (calib.isDefault) return entry; // zero-cost 최적화
  final oddsAdj = calib.userOddsWeight * kOddsWeightSpeedDelta * 333.0;
  final jockeyAdj = calib.userJockeyBuff * kJockeyBuffMaxDelta * 333.0;
  final newUserBonus = (entry.userBonus + oddsAdj + jockeyAdj).clamp(-5.0, 5.0);
  return entry.copyWith(userBonus: newUserBonus);
}
```

5-3. manual_calibration_panel.dart (UI 컴포넌트 핵심 발췌)

```
// — CalibrationOddsField: 배당률 가중치 입력창 —
TextFormField(
  keyboardType: TextInputType.numberWithOptions(decimal: true, signed: true),
  inputFormatters: [FilteringTextInputFormatter.allow(RegExp(r'^-?\wd{0,1}\w.\?\wd{0,1}$'))],
  onChanged: (v) {
    final parsed = double.tryParse(v);
    if (parsed != null) ctrl.setOddsWeight(widget.gateNo, parsed);
  },
)

// — CalibrationJockeySlider: 기수 멘탈 보정 슬라이더 —
Slider(
  value: val.clamp(kJockeyBuffMin, kJockeyBuffMax),
  min: -1.0, max: 1.0, divisions: 20,
  onChanged: enabled ? (v) => ctrl.setJockeyBuff(gateNo, v) : null,
)

// — CalibrationSpurtDropdown: 스퍼트 시점 드롭다운 —
DropdownButton<SpurtTimingSlot>(
  value: current,
  onChanged: enabled ? (slot) {
    if (slot != null) ctrl.setSpurtTiming(gateNo, slot);
  } : null,
  items: SpurtTimingSlot.values.map((slot) =>
    DropdownMenuItem(value: slot, child: Text(slot.label))
  ).toList(),
)
```

제6조 물리 엔진 연동 위치 명세 (race_animation_screen.dart)

6-1. _initHorses() — 경주 시작 시 1회 적용

```
void _initHorses() {
  // [UserCalib-1] oddsAdjFactor: baseSpeed 배당률 가중치 보정
  final calibratedBase = base * _calibCtrl.oddsAdjFactor(h.gateNo);
  // userOddsWeight +2.0 → baseSpeed × 1.030 (최대 +3%)
  // userOddsWeight -2.0 → baseSpeed × 0.970 (최대 -3%)

  // [UserCalib-2] spurtProgOffset: 스퍼트 시점 개별 오프셋 캐싱
  _horseSpurtOffset[h.gateNo] = _calibCtrl.spurtProgOffset(h.gateNo);
}
```

6-2. _onTick() → inZone4Straight — 매 틱 16ms 적용

```
if (inZone4Straight) {  
    // ... (기존 Jockey Engine 3종 적용) ...  
  
    // [UserCalib-3] speedMultOverride: 기수 멘탈 → Zone4 가속도 오버라이드  
    final userMultOverride = _calibCtrl.speedMultOverride(h.entry.gateNo, afScale);  
    if (userMultOverride != 1.0) {  
        speedMult *= userMultOverride;  
        // 버프 시 글로우 이펙트 추가  
        if (userMultOverride > 1.0 && !h.boostActive) {  
            h.boostGlow = (h.boostGlow + (userMultOverride - 1.0) * 0.5).clamp(0, 1);  
        }  
    }  
}
```

제7조 KRA 공공데이터 API26_2 (ENTRY SHEET) 연동 기술 명세

본 조는 한국마사회(KRA) 공공데이터포털이 제공하는 **API26_2 entrySheet_2** REST API의 응답 데이터가 본 저작물의 물리 엔진 파라미터에 어떻게 직접 대입되는지를 기술적으로 명세한다. 이는 "실제 경주 데이터를 실시간 물리 엔진에 직접 대입"하는 본 저작물의 핵심 독창성을 구성한다.

7-1. API26_2 서비스 개요

항목	내용
API명	한국마사회 출전표 상세정보 (entrySheet_2)
서비스 URL	https://apis.data.go.kr/B551015/API26_2/entrySheet_2
인터페이스	REST GET / XML 응답
데이터 갱신	수시 (경주 당일 출전표 확정 후)
적용 경주장	서울(meet=1), 제주(meet=2), 부산경남(meet=3)
응답 필드 수	50개 이상

7-2. 핵심 필드 → 물리 엔진 파라미터 직접 대입 매핑

아래 표는 API26_2 응답 필드가 본 저작물의 어떤 물리 엔진 파라미터에 직접 대입되는지를 명세한다. 이 직접 대입 구조 자체가 저작권 보호 대상인 독창적 알고리즘이다.

API26_2 필드명	국문명	물리엔진 파라미터	대입 로직
chulNo	마번(출주번호)	HorseEntry.gateNo	직접 대입, 레인 배치 결정
wgBudam	부담중량(kg)	HorseEntry.wgBudam	직접 대입 → kWgBudamPenalty 보정 인자
rating	레이팅	HorseEntry.rating	기반 스탯 4종 자동 계산
ord1CntT	통산1위횟수	—	rcCntT 와 함께 rcWins 직접 계산
rcCntT	통산출주횟수	HorseEntry.rcWins	$rcWins = ord1CntT / rcCntT$
ord1CntY	최근1년1위횟수	—	rcCntY 와 함께 최근 폼 계산
rcCntY	최근1년출주횟수	rcWins1Y (내부 지수)	$rcWins1Y = ord1CntY / rcCntY$
chaksun1	1착상금(원)	HorseEntry.prizeWin	경주 Prize Pool 표시
chaksun2	2착상금(원)	HorseEntry.prize2nd	경주 Prize Pool 표시
chaksun3	3착상금(원)	HorseEntry.prize3rd	경주 Prize Pool 표시
chaksun4	4착상금(원)	HorseEntry.prize4th	경주 Prize Pool 표시
chaksun5	5착상금(원)	HorseEntry.prize5th	경주 Prize Pool 표시
chaksunT	통산수득상금(원)	HorseEntry.prizeTotalCareer	AI 분석 참고 지수
chaksunY	최근1년수득상금(원)	HorseEntry.prizeTotal1Year	prizeCompetitiveness 산출 기반
chaksun_6m	최근6개월수득상금(원)	HorseEntry.prizeTotal6Month	g1fRating 근사 추정 기반
jkName	기수명	HorseEntry.jockeyName	Jockey Engine 매핑 키
hrNo	마번(고유번호)	HorseEntry.horseRegNo	고유 식별자

7-3. 승률 직접 계산 알고리즘 (독창적 구현)

기존 경마 정보 서비스는 별도 API에서 사전 계산된 승률 값을 수신한다. 본 저작물은 `ord1CntT` 와 `rcCntT` 두 원시 필드를 직접 수신하여 **앱 내부에서 승률을 직접 계산**하는 방식을 채택하였다. 이는 실시간 데이터 정합성을 높이는 독창적 구현이다.

```
// ——— 통산 경주마 승률 직접 계산 ———
// API26_2 필드: ord1CntT(통산1위횟수) / rcCntT(통산출주횟수) → rcWins
final ord1CntT = int.tryParse(item['ord1CntT']?.toString() ?? '0') ?? 0;
final rcCntT = int.tryParse(item['rcCntT']?.toString() ?? '0') ?? 0;
// 0으로 나누기 방지: 출주 이력 없으면 0.0
final rcWins = rcCntT > 0 ? (ord1CntT / rcCntT).clamp(0.0, 1.0) : 0.0;

// ——— 최근 1년 폼 승률 (최신 컨디션 반영) ———
// API26_2 필드: ord1CntY(최근1년1위횟수) / rcCntY(최근1년출주횟수)
final ord1CntY = int.tryParse(item['ord1CntY']?.toString() ?? '0') ?? 0;
final rcCntY = int.tryParse(item['rcCntY']?.toString() ?? '0') ?? 0;
final rcWins1Y = rcCntY > 0 ? (ord1CntY / rcCntY).clamp(0.0, 1.0) : 0.0;
```

7-4. 상금 기반 경쟁력 지수 (prizeCompetitiveness) 알고리즘

API26_2의 `chaksunY` (최근1년수득상금)는 단순 표시 데이터가 아닌, **AI 경쟁력 지수 산출의 직접 입력 파라미터**로 활용된다.

```
// ——— 상금 경쟁력 지수 (0.0 ~ 1.0 정규화) ———
// HorseEntry getter — 최근 1년 상금 기반 경쟁력 지수
// 1억(100,000,000)원 기준으로 정규화
double get prizeCompetitiveness =>
    (prizeTotal1Year / 100000000.0).clamp(0.0, 1.0);

// ——— baseScore 계산에 상금 지수 반영 ———
final prizeComp = (prizeTotal1Year / 100000000.0).clamp(0.0, 1.0);
final prizebonus = prizeComp * 5.0; // 상금 경쟁력 최대 +5점

final baseScore = (
    speedStat * 0.35 +
    staminaStat * 0.25 +
    formStat * 0.20 +
    trackFitStat * 0.10 +
    rating * 0.07 +
    prizebonus * 0.03 // ← API26_2 chaksunY 직접 반영 (3%)
).clamp(0.0, 100.0);
```

7-5. KRA API 에러 코드 전체 핸들링 (`KraApiErrorCode`)

본 저작물은 KRA 공공데이터포털 공식 명세서에 명시된 9개 에러코드 전체를 처리하는 `KraApiErrorCode` 클래스를 구현하였다. 이는 서비스 안정성과 사용자 경험을 위한 독창적 방어 코드이다.

```

class KraApiErrorCode {
    // KRA 공식 에러코드 전체 (API26_2 명세서 기반)
    static const String applicationError = '1'; // 어플리케이션 에러
    static const String invalidParam = '10'; // 잘못된 요청 파라미터
    static const String noService = '12'; // 서비스 없거나 폐기됨
    static const String accessDenied = '20'; // 서비스 접근 거부
    static const String requestLimitExceeded = '22'; // 요청 제한 횟수 초과
    static const String serviceKeyNotReg = '30'; // 미등록 서비스키
    static const String serviceKeyExpired = '31'; // 기한 만료 서비스키
    static const String unregisteredIp = '32'; // 미등록 IP
    static const String unknown = '99'; // 기타 에러
    static const String normal = '00'; // 정상

    // 에러 분류 메서드
    static bool isAuthError(String code) =>
        code == '20' || code == '30' || code == '31' || code == '32';
    static bool isRetryable(String code) =>
        code == '1' || code == '10' || code == '22' || code == '99';
    static bool isPermanent(String code) => code == '12';
}

```

에러 처리 전략:

에러 코드	유형	처리 방식
00	정상	reportServerOk() → 장애 배너 해제
1, 99	일시적	reportServerError() → 10분 주기 재시도
10	파라미터 오류	reportServerError() → 재시도
12	서비스 폐기	reportServerError() → 영구 장애 표시
20, 30, 31, 32	인증 오류	reportServerOk() → 서버 정상 판단 (키 문제)
22	요청 한도 초과	reportServerError() → 재시도

제8조 KRA racedetailresult (경주별상세성적표) 연동 기술 명세

본 조는 한국마사회 `racedetailresult/getracedetailresult` REST API(XML 응답)의 전체 응답 필드가 경주결과 화면(`RaceResultScreen`)에 어떻게 직접 매핑되는지를 기술한다.

8-1. racedetailresult 서비스 개요

항목	내용
API명	한국마사회 경주별상세성적표 (<code>racedetailresult</code>)
Call Back URL	<code>http://apis.data.go.kr/B551015/racedetailresult/getracedetailresult</code>
인터페이스	REST GET / XML 전용 (JSON 미지원)
최대 응답 크기	4,000 bytes
평균 응답 시간	500 ms
초당 최대 TPS	30
데이터 갱신	일1회 (경주 종료 후 집계)

8-2. 응답 필드 전체 → HorseResult 모델 직접 매핑

API 필드명	국문명	HorseResult 필드	활용 방법
stOrd	착순	rank	착순 표시, 정렬 키
chulNo	출주번호	gateNo	마번 배지, 레인 색상
chulYn	출전여부 (1=출전, 0=미출전)	didStart	미출전마 별도 섹션 분리
hrNo	마번(고유등록번호)	horseNo	말 개별 식별자
hrName	마명	horseName	말이름 표시
meet	경마장	venueName	경마장명
prdCtyNm	산지 (한/외)	origin	마필 정보
sex	성별 (거/牡/牝)	sex	마필 정보
age	연령	age	마필 정보
wgHr	마체중(kg)	weight	체중 표시
df	마체중편차(kg)	weightDiff	체중 증감 표시
wgBudam	부담중량(kg)	wgBudam	기수 부담중량
hrTool	장구내역	horseTool	장구 뱃지 UI (보라색 태그)
hrRating	경주마레이팅	horseRating	레이팅 참고
jkName	기수명	jockeyName	기수명 표시
jkNo	기수번호	jockeyNo	기수 식별자
jkMeet	기수경마장	jockeyMeet	소속 경마장
jkSymbol	수습기수감량(kg)	jockeyApprentice	"수습" 뱃지 + 감량 표시
trName	조교사명	trainerName	조교사 정보
trNo	조교사번호	trainerNo	식별자
trMeet	조교사경마장	trainerMeet	소속 경마장
owName	마주명	ownerName	마주 정보
owNo	마주번호	ownerNo	식별자
owCloth	마주복식	ownerCloth	실크 패턴
rcTime	경주기록	raceTime	주파기록 표시 (1착 강조)
differ	도착차	differ	착차 표시 (1착 제외)

API 필드명	국문명	HorseResult 필드	활용 방법
win	단승식배당율	winOdds	단승 배당 칩 (주황)
plc	연승식배당율	placeOdds	연승 배당 칩 신규 추가 (파랑)

8-3. XML 정규식 파싱 알고리즘 (독창적 구현)

racedetailresult는 JSON을 지원하지 않고 **XML 전용**으로 응답한다. 본 저작물은 별도 XML 파서 라이브러리 없이 **정규식 기반 경량 파싱 엔진**을 자체 구현하였다.

```
// — XML <item> 블록 추출 —
static List<Map<String, String>>? _extractXmlItems(String xml) {
    // 1단계: resultCode 검증
    final codeMatch = RegExp(r'<resultCode>(Wd+)</resultCode>').firstMatch(xml);
    final code = codeMatch?.group(1) ?? '';
    if (code.isEmpty() && code != '00') return null; // 에러코드 처리

    // 2단계: <item>...</item> 블록 추출 (DOTALL 모드)
    final itemMatches = RegExp(
        r'<item>([WsWS]*?)</item>',
        multiline: true,
    ).allMatches(xml);

    // 3단계: 각 블록에서 <tag>value</tag> 파싱
    return itemMatches.map((m) {
        final block = m.group(1) ?? '';
        final map = <String, String>{};
        final tagMatches = RegExp(r'<(Ww+)>([WsWS]*?)</Ww+>').allMatches(block);
        for (final t in tagMatches) {
            map[t.group(1)!] = t.group(2)!.trim();
        }
        return map;
    }).toList();
}
```

8-4. 미출전마(chulYn=0) 처리 알고리즘

chulYn=0 인 말은 착순 정렬에서 분리하여 별도 UI 섹션으로 표시한다.

```
// 착순 정렬: 출전마 → 미출전마 순서
results.sort((a, b) {
    if (a.didStart && !b.didStart) return -1; // 출전마 우선
    if (!a.didStart && b.didStart) return 1;
    if (a.didStart && b.didStart) return a.rank.compareTo(b.rank); // 착순
    return a.gateNo.compareTo(b.gateNo); // 미출전마: 마번순
});

// UI: 출전마 테이블 아래 별도 "출전취소(미출전)" 섹션 렌더링
final scratched = horses.where((h) => !h.didStart).toList();
if (scratched.isNotEmpty) _buildScratchedSection(scratched);
```

8-5. 9999.9 배당 처리 (비대상 배당 필터링)

KRA API는 단승/연승 배당 비대상 말에 9999.9를 반환한다. 본 저작물은 이를 0.0으로 변환하여 UI에서 -로 표시한다.

```
double parseOdds(String? s) {
    final v = double.tryParse(s ?? '0') ?? 0.0;
    return v >= 9999.0 ? 0.0 : v; // 9999.9 → 0.0 변환
}
```

8-6. KraApiErrorCode 공식 영문 메시지명 (v1.2.0 추가)

```
// 에러코드 → 공식 영문 에러메시지명 (OpenAPI 명세서 기준)
static String toEnglishMessage(String code) {
    switch (code) {
        case '1': return 'APPLICATION_ERROR';
        case '10': return 'INVALID_REQUEST_PARAMETER_ERROR';
        case '12': return 'NO_OPENAPI_SERVICE_ERROR';
        case '20': return 'SERVICE_ACCESS_DENIED_ERROR';
        case '22': return 'LIMITED_NUMBER_OF_SERVICE_REQUESTS_EXCEEDS_ERROR';
        case '30': return 'SERVICE_KEY_IS_NOT_REGISTERED_ERROR';
        case '31': return 'DEADLINE_HAS_EXPIRED_ERROR';
        case '32': return 'UNREGISTERED_IP_ERROR';
        case '99': return 'UNKNOWN_ERROR';
        case '00': return 'NORMAL SERVICE';
        default: return 'UNKNOWN_ERROR';
    }
}
```

제9조 버전 이력

버전	날짜	변경 내용
v1.0.0	2025-05-22	초기 구현: UserCalibrationController, HorseCalibration, ManualCalibrationPanel
v1.0.0	2025-05-22	race_animation_screen.dart 연동: oddsAdjFactor, speedMultOverride, spurtProgOffset
v1.0.0	2025-05-22	flutter analyze 0 errors 확인 완료
v1.1.0	2025-05-23	KRA API26_2 파싱 전면 개선: ord1CntT/rcCntT 승률 직접 계산 구현
v1.1.0	2025-05-23	HorseEntry 상금 필드 8종 추가 (prizeWin~prizeTotal6Month)
v1.1.0	2025-05-23	KraApiErrorCode 클래스 신규 — 9개 공식 에러코드 전체 핸들링
v1.1.0	2025-05-23	prizeCompetitiveness 상금 경쟁력 지수 → baseScore 3% 반영
v1.1.0	2025-05-23	g1fRating 최근 6개월 상금 기반 근사 추정 로직 추가
v1.2.0	2025-05-23	racedetailresult API 전면 연동: HorseResult 26개 필드 완전 매핑
v1.2.0	2025-05-23	XML 정규식 파싱 엔진 자체 구현 (_extractXmlItems)
v1.2.0	2025-05-23	chulYn 미출전마 별도 UI 섹션 분리
v1.2.0	2025-05-23	jkSymbol 수습기수 감량 배지 UI 추가
v1.2.0	2025-05-23	hrTool 장구내역 배지 UI 추가
v1.2.0	2025-05-23	differ 도착차 + plc 연승배당 테이블 행 신규 표시
v1.2.0	2025-05-23	9999.9 배당 → 0.0 변환 parseOdds() 함수
v1.2.0	2025-05-23	KraApiErrorCode.toEnglishMessage() 공식 영문명 추가

본 문서는 한국저작권위원회 컴퓨터프로그램저작물 등록 신청을 위해 작성된 기술 명세서입니다.

소스코드의 무단 복제, 배포, 역공학 분석을 금합니다.

Copyright © 2025 경마통 Race Master. All rights reserved.